Learn Functional Programming with Elixir

Introduction

- Run `elixir -v` to check version and `iex` for an interactive REPL. Hit Ctrl+C twice to quickly exit the REPL. It has tab completion.
- End script files with `.exs` and compiled files with `.ex`. It's just `elixir hello_world.exs` to run a script.

Chapter 1 - Thinking Functionally

- In the functional programming paradigm, functions are the basic building blocks, all values are immutable, and the code is declarative.
- Immutable data structures can be shared by concurrent processes without locks. It's easy to make mistakes when a language has mutability as default.
- The complexity of building a larger application is reduced when it is composed of functions with these properties (called *pure functions*):
  - The values are immutable.
  - The function's result is only affected by the function's arguments.
  - The function doesn't generate effects beyond the value it returns.
- In Elixir the focus is on data-transformation flow, and it has a special operator called pipe (`|>`) to combine multiple functions calls together.

```
def capitalize_words(title) do
  title
  |> String.split
  |> capitalize_all
  |> join_with_whitespace
end
```

- Declarative programming focuses on what is necessary to solve a problem, describing the data flow.

```
defmodule StringList do
  def upcase([]), do: []
  def upcase([first | rest]), do: [String.upcase(first) | upcase(rest)]
end

StringList.upcase(["dogs", "hot dogs", "bananas"])
```

Chapter 2 - Working with Variables and Functions

- `string`, `integer`, `float`, and `boolean` are as expected.
- The `atom` type is a constant and its name is the value: `:ok, :error`.
- `true`, `false`, and `nil` are the atoms `:true`, `:false`, and `:nil`.
- Tuples are fixed size and use braces: `{:ok, "Hello"}, {1, 2, 3}`
- maps (dictionaries) start with a percent (%): `%{id: 123, name: "Anna"}` or `%{"name" => "Alice", "age" => 23}`, `%{:name => "Alice", :age => 30}`, or even `%{name: "Alice", age: 30}` when all keys are atoms.
- Null is known as `nil` or even `:nil`.
- `++` concatenates lists; `<>` concatenates strings or binaries.
- `and` / `or` require booleans on their left side, and `not` takes booleans
- `&&` / `||` / `!` accept truthy falsy values on their left side.
- `false` and `nil` are falsy; everything else is truthy.
- `&&` returns the second expression's value when the first is truthy and otherwise the first. `||` returns the first truthy expression or the last expression if none are truthy. `!` returns `true` when the value is falsy and `false` when the value is truthy. You can often use these operators in short circuit expressions like `cache_image || fresh_image`.
- Variables can be re-assigned (they are `var` not `val`).
- Variables follow snake_case naming conventions. Variable names cannot start with a capital letter, and it is a "match error" to try. Names that start with a capital letter are used in modules.

- You need a `.` to call anonymous functions but not those in a module:

```
get_double = fn x -> x * 2 end
get_double.(5) # <--- Notice the dot!
String.upcase("hello") # Module.function(arguments)
String.upcase "hello"  # parentheses are also optional.
```

- You can do string interpolation with `#{}`:

```
first_name = "Lewis"
last_name = "Cawthorne"
full_name = "#{first_name} L #{last_name}"
hello = fn name -> "Hello, #{name}!" end
hello.("World!") # "Hello, World!"
```

- You can put two commands on the same line with the semicolon (;).
- Closures remember all the free variables that were referenced in the lexical scope in which they were created.
- Common Elixir modules: `String`, `Integer`, `Float`, `IO`, `Kernel` with `String.capitalize("hI Friends!")`, `String.downcase("OW")`, `Integer.parse("123")`, `Integer.to_string(-890)`, `Integer.digits(890)`, `Float.ceil(3.7)`, `Float.floor(3.7)`, `Float.round(3.7576, 2)`, `IO.puts("Hello, World!")`, `IO.gets("What's your name?")`, `IO.inspect({:ok, 123})`, `div(1, 2)`, `rem(1, 2)`, `is_number("Hi")`.
- Functions from `Kernel` are available without the module name.
- Modules go in `ex` files and begin with `defmodule`:

```
# checkout.ex
defmodule Checkout do
  def total_cost(price, tax_rate) do
    price * (tax_rate + 1)
  end
end
```

- To use this module in IEx, run `c("checkout.ex")` then `Checkout.total_cost(100, 0.2)`. The `c` function compiles a module and makes it available to the current IEx session.

- You could've defined `total_cost` on one line like:

```
defmodule Checkout do
  def total_cost(price, tax_rate), do: price * (tax_rate + 1)
end
```

- Functions return the value of the last expression.
- Functions and files are named like variables, `snake_case`. Modules are `CamelCase`. The module filename would be `camel_case.ex`. Module filenames match module names except are lowercase and snake_case. You define one module per file. The modules go in a directory with the name of the module's namespace (`Ecommerce.Checkout` in `./ecommerce/checkout.ex`).
- In a new application, it's a good practice to put a name, a *namespace*, before each of the custom modules names separated by dots. `defmodule Ecommerce.Checkout do`.
- We can call any function with `ModuleName.name_of_the_function`, but if you don't want to type `ModuleName` you should use `import`.

```
defmodule TaskListWithImport do
  import File, only: [write: 3, read: 1]

  @file_name "task_list.md"

  def add(task_name) do
    task = "[ ] " <> task_name <> "\n"
    write(@file_name, task, [:append])
  end

  def show_list do
    read(@file_name)
  end
end
```

- In the above example, `@file_name` is a *module attribute*. Module attributes can be used as annotations, temporary storage, or constants. Here we're using it as a constant. It will be available the entire module.
- When importing a named function, we must always pass its name and arity.

- `import File` without `only` will import all functions in `File` module.
- We can get a reference to a function like `&String.upcase/1`
- We can also use `&` to declare anonymous functions: `total_cost = &(&1 * &2)`. `&` creates the anonymous function and `&1` and `&2` refer to the first and second arguments. We can't use anonymous syntax for a function with zero arity, so use `fn -> true end` instead. The parentheses are even optional: `total_cost = & &1 * &2`.

```
Enum.map(["hello", "world"], &String.upcase/1)
multiplier = fn x, y -> x * y end
multiplier = & &1 * &2
```

- Don't go capture syntax crazy with anonymous functions as it makes for hard to read code.

Chapter 3 - Using Pattern Matching to Control the Program Flow

- `=` tries to pattern match and raises a `MatchError` if it fails. `2 = x` fails with a `MatchError` unless the value of `x` is 2. `^x = 2` also fails with a `MatchError` unless `x` is 2. The pin operator (`^`) avoids rebinding and uses the value of `x` to pattern match. Otherwise a variable on the left of equals is rebound.
- Destructuring is our primary tool to get a string part, an item from a list, or a value from a map.

```
# You can pattern match with String parts, but you must not have a variable
# to the left of the <> operator.
"Authentication: " <> credentials = "Authentication: Basic dXNlcjpwYXNz"
"Basic dXNlcjpwYXNz" = credentials
"eoD " <> first_name = String.reverse("John Doe")
"John" = String.reverse(first_name)
{a, b, c} = {4, 5, 6} # 4 = a; 5 = b; 6 = c
process_life_the_universe_and_everything = fn -> {:ok, 42} end
{:ok, answer} = process_life_the_universe_and_everything.()
IO.puts "The answer is #{answer}."
```

- It is a common but not universal convention to return tuples of `{:ok, value}` for success and `{:error, :error_type}` for errors.

```
user_input = IO.gets "Write your ability score:\n"
{ability_score, _} = Integer.parse(user_input)
ability_modifier = (ability_score - 10) / 2
IO.puts "Your ability modifier is #{ability_modifier}"
```

- On success, `Integer.parse` returns `{parsed_number, remaining_text}`. On failure, it returns `:error`.
- `=` is the match operator - it only does pattern matching, but matching a bare variable against a value looks like assignment. `true` when arguments are equivalent and have the same type.
- In Elixir, lists are *linked lists*. The last item is an empty list (`[]`).
- A variable has a unique value in an expression, so:

```
[a, a, "pineapples"] = ["apples", "apples", "pineapples"]
[a, a, a] = [1, 2, 1] # MatchError
[_, a, _] = [10, 2, 12] # a == 2
[ head | tail ] = [:a, :b, :c, :d] # head == :a; tail == [:b, :c, :d]
[ head | tail ] = [:a] # head == :a; tail == []
[ head | tail ] = [] # MatchError
[ a, b | rest] = [1, 2, 3, 4] # a == 1;b ==2; rest==[3, 4]
```

- The wildcard (`_`) matches anything and ignores the value.
- The `|` operator matches the first item(s) of a list, values to the right always matches the rest of the elements.
- A map pattern match checks a subset of a map, so you only need to provide keys your interested in. If the keys don't exist, a `MatchError` will be raised.

```
abilities = %{strength: 16, dexterity: 12, intelligence: 10}
%{strength: strength_value} = abilities # strength_value == 16
%{wisdom: wisdom_value} = abilities # MatchError
%{} = abilities # true; an empty match matches any map
# below checks int is 10 and binds 12 to dexterity_value
%{intelligence: 10, dexterity: dexterity_value} = abilities
%{strength: strength_value = 16} = abilities # assigns 16 since it is 16
```

- A keyword list is a list of two-element tuples: it allows duplicated keys but they must be atoms.

```
[b, c] = [a: 1, a: 12] # b == {:a, 1}; c == {:a, 12}
# the below uses a keyword list to import two functions of different arity.
import String, only: [pad_leading: 2, pad_leading: 3]
```

- Structs are useful for consistent structures that have the same set of keys everywhere in the application. All structs have a list of permitted attributes and these are guaranteed by the compiler. All `Date` structs have `year`, `month`, `day`, and `calendar`. You could not define `hot_dog` in a `Date` struct. Pattern matching works for them similar to maps.

```
date = ~D[2018-01-01]
%{year: year} = date # year == 2018
%Date{day: day} = date # day == 1; we used the struct type to match.
%Date{day: day} = %{day: 1} # MatchError. Not a Date.
```

- You can also use pattern matching to define functions. When you define a function using pattern matching, you define a series of *function clauses*. They need to be defined in sequence and are checked top to bottom.

```
defmodule NumberCompare do
  def greater(number, other_number) do
    check(number >= other_number, number, other_number)
  end

  defp check(true, number, _), do: number
  defp check(false, _, other_number), do: other_number
end
```

- `defp` defines private functions of your module. Other modules don't need access to `check` in order to use `greater`.

- We can define a default value for an argument with `\\`. You can only have one default value for a particular parameter.

```
# checkout.ex
defmodule Checkout do
  def total_cost(price, quantity \\ 10), do: price * quantity
end

iex> c("checkout.ex")
iex> Checkout.total_cost(12) # 120
iex> Checkout.total_cost(12, 5) # 60
```

- Guard clauses allow us to add Boolean expressions to our functions.

```
defmodule NumberCompare do
  def greater(number, other_number) when number >= other_number, do: number
  def greater(_, other_number), do: other_number
end

defmodule Checkout do
  # If you call this with negative values, you get FunctionClauseError.
  # If you call this with a string, you get ArithmeticError.
  def total_cost(price, tax_rate) when price >= 0 and tax_rate >= 0 do
    price * (tax_rate + 1)
  end
end
```

- Only certain functions and operators can be used in guard clauses. You can use `defguard` to define guard clauses for your functions.

```
defmodule Checkout do
  defguard is_rate(value) when is_float(value) and value >= 0 and value <= 1
  defguard is_cents(value) when is_integer(value) and value >= 0

  def total_cost(price, tax_rate) when is_cents(price) and is_rate(tax_rate)
do
    price + tax_cost(price, tax_rate)
  end

  def tax_cost(price, tax_rate) when is_cents(price) and is_rate(tax_rate) do
    price * tax_rate
  end
end
```

- `case` is use when we want to check an expression with multiple pattern-match clauses. `case` evaluates to the last expression.

```elixir
user_input = IO.gets "Write your ability score:\n"
case Integer.parse(user_input) do
  :error -> IO.puts "Invalid ability score: #{user_input}"
  {ability_score, _} ->
    ability_modifier = (ability_score - 10) / 2
    IO.puts "Your ability modifier is #{ability_modifier}"
end

# alternatively
result = case Integer.parse(user_input) do
  :error ->
    "Invalid ability score: #{user_input}"
  {ability_score, _} ->
    ability_modifier = (ability_score - 10) / 2
    "Your ability modifier is #{ability_modifier}"
end
IO.puts result

# or even
result = case Integer.parse(user_input) do
  :error ->
    "Invalid ability score: #{user_input}"
  {ability_score, _} when ability_score >= 0 ->
    ability_modifier = (ability_score - 10) / 2
    "Your ability modifier is #{ability_modifier}"
end
```

- If no line matches a `case` clause, an error will be raised and your process will stop.
- `cond` is useful when you want to check different variables and values in logical expressions and don't need pattern matching for the problem.

```elixir
{age, _} = Integer.parse IO.gets("Person's age:\n")

result = cond do
  age < 13 -> "kid"
  age <= 18 -> "teen"
  age > 18 -> "adult"
end

IO.puts "Result: #{result}"
```

- In a `cond` statement the code associated with the first condition that evaluates to something truthy will be run. It raises an error on no match.
- `if` is useful if you want to execute a command when some expression results in a truthy value. `unless` is similar but the `unless` block is executed when the expression is `nil` or `false`. Using `unless` with an `else` clause can be confusing, so use `if` instead.

```elixir
defmodule NumberCompareWithIf do
  def greater(number, other_number) do
    if number >= other_number do
      number
    else
      other_number
    end
  end
end

# You can also do a single line if.
result = if(number >= other_number, do: number, else: other_number)

defmodule NumberCompareWithUnless do
  def greater(number, other_number) do
    unless number < other_number do
      number
    else
      other_number
    end
  end
end
```

- `if` and `unless` return the resulting value of the executed code block. `else` is optional, and it returns `nil` with an omitted `else`.

Chapter 4 - Diving into Recursion

- Recursive functions are the core of repetition in functional programming.

```
# Simple bounded recursion.
defmodule Sum do
  def up_to(0), do: 0
  def up_to(n), do: n + up_to(n - 1)
end

# List pattern matching.
defmodule Math do
  def sum([]), do: 0
  def sum([head | tail]), do: head + sum(tail)
end
```

- You can use the `|` operator to construct a new list.

```elixir
defmodule EnchanterShop do
  def test_data do
    [
      %{title: "Longsword", price: 50, magic: false},
      %{title: "Healing Potion", price: 60, magic: true},
      %{title: "Rope", price: 10, magic: false},
      %{title: "Dragon's Spear", price: 100, magic: true}
    ]
  end

  @enchanter_name "Edwin"

  def enchant_for_sale([]), do: []
  def enchant_for_sale([item = %{magic: true} | incoming_items]) do
    [item | enchant_for_sale(incoming_items)]
  end
  def enchant_for_sale([item | incoming_items]) do
    new_item = %{
      title: "#{@enchanter_name}'s #{item.title}",
      price: item.price * 3,
      magic: true
    }

    [new_item | enchant_for_sale(incoming_items)]
  end
end
```

- You can access map elements as `item[:title]` or `item.title`. The latter only works for atoms, but `item["title"]` for example is fine for strings. Bracket access returns `nil` for a missing key, but dot-access requires the key to exists or else it throws a `KeyError`.
- In recursion, you can decrease and conquer or divide and conquer. Decrease and conquer starts from the base case. Divide and conquer is about separating the problem into two or more parts that can be processed independently and combined at the end.
- To use tail-call optimization, we need to make sure the last expression of our function is a function call. An easy way to do this is to replace the use of the function result with an extra argument that accumulates the results of each iteration.

```elixir
defmodule TRFactorial do
  def of(n), do: factorial_of(n, 1)
  defp factorial_of(0, acc), do: acc
  defp factorial_of(n, acc) when n > 0, do: factorial_of(n - 1, n * acc)
end
```

- You can limit unbounded recursion with a guard of a parameter like depth.
- You can create a recursive anonymous function by wrapping the anonymous function in another call that passes in itself:

```elixir
fact_gen = fn me ->
  fn
    0 -> 1
    x when x > 0 -> x * me.(me).(x - 1)
  end
end
factorial = fact_gen.(fact_gen)
factorial.(5) # 120
factorial.(10) # 3628800
```

- A cleaner example from Gemini:

```elixir
# Calculating a factorial with an anonymous recursive function:
factorial = fn
  0, _self -> 1
  n, self -> n * self.(n - 1, self)
end

# To run it, you have to pass `factorial` into itself as the 2nd argument:
factorial.(5, factorial) #=> 120
```

- It's a neat trick that you can make recursive anonymous functions but you will seldom see it in the wild.

Chapter 5 - Using Higher-Order Functions

- We designed a lot of our own recursive list functions in `ch05/my_list.ex`.

- `map`, `reduce`, and `filter` are all defined in `Enum`. so are `count`, `uniq`, `sum`, `sort`, `member?`, and `join`.

```
Enum.count(["dogs", "cats", "flowers"]) # 3
Enum.uniq(["a", "a", "b", "b", "b", "c"]) # ["a", "b", "c"]
Enum.sum([10, 5, 5, 10]) # 30
Enum.sort(["c", "b", "d", "a"], &<=/2) # ["a", "b", "c", "d"]
Enum.sort(["c", "b", "d", "a"], &>=/2) # ["d", "c", "b", "a"]
Enum.member?([10, 20, 12], 10) # true
Enum.join(["apples", "hot dogs", "flowers"], ", ")
# "apples, hot dogs, flowers"
```

- `Enum` functions work with any data type that respects Enumerable protocol.

```
upcase = fn {_key, value} -> String.upcase(value) end
Enum.map(%{name: "willy", last_name: "wonka"}, upcase) # ["WONKA", "WILLY"]

# And `Enum` has functions that take more than one function.
medals = [
  %{medal: :gold, player: "Anna"},
  %{medal: :silver, player: "Joe"},
  %{medal: :gold, player: "Zoe"},
  %{medal: :bronze, player: "Anna"},
  %{medal: :silver, player: "Anderson"},
  %{medal: :silver, player: "Peter"},
]
# Now let's show the players that have won each type of medal.
# In other words, we will group by medal and show player names in each group.
Enum.group_by(medals, &(&1.medal), &(&1.player))
# %{bronze: ["Anna"], gold: ["Anna", "Zoe"], silver: ["Joe", "Anderson",
"Peter"]}
```

- Elixir also has list comprehensions using the `for` keyword:

```elixir
for a <- ["dogs", "cats", "flowers"], do: String.upcase(a)
# ["DOGS", "CATS", "FLOWERS"]
for a <- ["Willy", "Anna"], b <- ["Math", "English"], do: {a, b}
[{"Willy", "Math"}, {"Willy", "English"}, {"Anna", "Math"}, {"Anna",
"English"}]
parseds = for i <- ["10", "hot dogs", "20"], do: Integer.parse(i)
# [{10, ""}, :error, {20, ""}]
for {n, _} <- parseds, do: n # [10, 20]
for n <- [1, 2, 3, 4, 5, 6, 7], n > 3, do: n # [4, 5, 6, 7]
```

- Elixir doesn't have a built-in composition operator, but you can create it yourself using the pipe operator (|>).

```elixir
# First let's try traditional function composition
defmodule HigherOrderFunctions do
  def compose(f, g) do
    fn arg -> f.(g.(arg)) end
  end
end
first_letter_and_upcase = compose(&String.upcase/1, &String.first/1)
first_letter_and_upcase("works") # W
# Now let's see the same with pipe.
first_letter_and_upcase = &(&1 |> String.first |> String.upcase)
first_letter_and_upcase("works") # W
```

- The pipe operator evaluates the expression before |> and passes it to the next function call as the first parameter and shines for combining more than two functions.

```elixir
# Imagine the following series of nested functions.
def capitalize_words(title) do
  Enum.join(
    Enum.map(
      String.split(title),
      &String.capitalize/1
    )
  )
end
capitalize_words("a whole new world") # "A Whole New World"
# Let's rewrite capitalize_words/1 the Elixir way.
def capitalize_words(title) do
  title
  |> String.split
  |> Enum.map(&String.capitalize/1)
  |> Enum.join(" ")
end
capitalize_words("a whole new world") # "A Whole New World"
# You could also define this with a series of descriptive helper functions.
def capitalize_words(title) do
  title
  |> String.split
  |> capitalize_all
  |> join_with_whitespace
end
def capitalize_all(words) do
  Enum.map(words, &String.capitalize/1)
end
def join_with_whitespace(words) do
  Enum.join(words, " ")
end
capitalize_words("a whole new world") # "A Whole New World"
```

- When calling functions in a pipeline that take multiple parameters, be sure to call them with parentheses so this doesn't error out.
- Elixir supports partial application through anonymous functions:

```elixir
defmodule WordBuilder do
  def build(alphabet, positions) do
    # String.at/2 requires two arguments, a string and a position.
    letters = Enum.map(positions, String.at(alphabet)) # won't work
    Enum.join(letters)
  end
end
# But we can write this with partial application to set String.at/2's
# first argument:
def build(alphabet, positions) do
  # We set the first parameter and it still takes the second one.
  partial = fn at -> String.at(alphabet, at) end
  letters = Enum.map(positions, partial)
  Enum.join(letters)
end
WordBuilder.build("world", [4, 1, 1, 2]) # "door"
# We could've also written that anonymously, which is common.
def build(alphabet, positions) do
  letters = Enum.map(positions, &(String.at(alphabet, &1)))
  Enum.join(letters)
end
```

- In Elixir we have the *streams* type for a flow of data that may not have an end and the `Stream` module that contains higher-order functions to operate on and create our streams.
- Elixir has a range literal that is the simplest stream: `1..10`. Streams are lazy. Consider this definition of factorial with streams:

```elixir
defmodule Factorial do
  def of(0), do: 1
  def of(n) when n > 0 do
    1..10_000_000
      |> Enum.take(n)
      |> Enum.reduce(&(&1 * &2))
  end
end
Factorial.of(5) # Only evaluates first five numbers of stream.
```

- One problem is it doesn't work for numbers larger than 10_000_000, so we can do better. `Stream.iterate/2` takes a starting number and an increment function and generates an infinite list.

```
integers = Stream.iterate(1, fn previous -> previous + 1 end)
Enum.take(integers, 5) # [1, 2, 3, 4, 5]
defmodule Factorial do
  def of(0), do: 1
  def of (n) when n > 0 do
    Stream.iterate(1, &(&1 + 1))
      |> Enum.take(n)
      |> Enum.reduce(&(&1 * &2))
  end
end
Factorial.of(5) # 120
```

- `Stream.cycle/1` will generate an infinite list that cycles over the same items.

```
defmodule Halloween do
  def give_candy(kids) do
    ~w(chocolate jelly mint)
    |> Stream.cycle
    |> Enum.zip(kids)
  end
end
Halloween.give_candy(~w(Mike Anna Ted Mary Alex Emma))
# [{"chocolate", "Mike"}, {"jelly", "Anna"}, {"mint", "Ted"},
#  {"chocolate", "Mary"}, {"jelly", "Alex"}, {"mint", "Emma"}]
```

- `~w` is the sigil for word list. `Enum.zip/2` pairs items from two lists until one of the lists runs out. `Stream.zip/2` would be useful for a lazy combination.

- An Elixir pipeline composed of `Enum` calls will be eager. Most functions are eager. But we can instead compose a lazy pipeline with Elixir streams. `Stream.map/2` will process items one at a time in a pipeline, and `Stream.chunk_every/2` combined with `Stream.flat_map/2` will process in chunks at a time. This is very useful when you have a collection of tasks that may take some time and don't want to leave the consumer at the end waiting.

Chapter 6 - Designing Your Elixir Applications

- Start a new app with `mix new appname`.

- You can add a new Mix task by creating a new file in the directory `lib/mix/tasks`. A simple Mix.Task:

```
defmodule Mix.Tasks.Start do
  use Mix.Task

  def run(_), do: IO.puts "Hello, World!"
end
```

- When building Elixir applications it's a good practice to put all the modules and related code under your application domain namespace. We define our character struct in lib/dungeon_crawl/character.ex in DungeonCrawl namespace.

- You use `defstruct` to define a character struct, passing a keyword list. The key is the attribute name, and the value will be the default value.

```
defmodule DungeonCrawl.Character do
  defstruct name: nil,
            description: nil,
            hit_points: 0,
            max_hit_points: 0,
            attack_description: nil,
            damage_range: nil
end
```

- Run `iex -S mix` to start a shell and load all modules automatically.

- `alias DungeonCrawl.Character` lets you refer to it as `%Character`. You can also `alias Mix.Shell.IO, as: Shell`.

- `Shell.info(message)` displays output, `Shell.yes?(prompt)` confirms Yn, `Shell.prompt(message)` prompts Enter, and `Shell.cmd(command)` runs commands.

- Elixir's protocol feature lets you create a single interface that various data types can implement. Using that, you can have polymorphism. You define a protocol with `defprotocol` and `def` functions without bodies. You implement the protocol with `defimpl`.

```
defprotocol DungeonCrawl.Display do
  def info(value)
end

defimpl DungeonCrawl.Display, for: DungeonCrawl.Room.Action do
  def info(action), do: action.label
end

defimpl DungeonCrawl.Display, for: DungeonCrawl.Character do
  def info(character), do: character.name
end
```

- You can also implement `to_string/1` for `String.Chars`.

```
defimpl String.Chars do
  def to_string(character), do: character.name
end
```

- The convention for protocol code organization is if you own the Struct, put the implementation in the same file as the struct. If you don't own the struct but own the protocol, put the implementation inside of the protocol file. If you own neither, create a file with the protocol name and put the implementation there.

- A *behaviour* is a contract between a module and the client code that provides a common interface for a client across multiple modules. For example `Mix.Task` is a behaviour and when we create a module that follows the `Mix.Task` behaviour we must implement the `run/1` function. Having multiple functions respect the same behaviour permits us to have a central point of execution and handling.

- The following declares that any module that obeys the contract must have a run function that takes two arguments of any type and returns a value of any type:

```elixir
defmodule DungeonCrawl.Room.Trigger do
  @callback run(character :: any, action :: any) :: any
end

# And in another file we implement the behavior
defmodule DungeonCrawl.Room.Triggers.Exit do
  @behaviour DungeonCrawl.Room.Trigger
  def run(character, _), do: {character, :exit}
end
```

- Elixir supports type specifications and the Dialyzer tool with static check to verify if type usage is correct. It is common to use `t` for the struct type. Ex:

```elixir
defmodule DungeonCrawl.Character do
  @type t :: %DungeonCrawl.Character{
    name: String.t,
    description: String.t,
    hit_points: non_neg_integer,
    max_hit_points: non_neg_integer,
    attack_description: String.t,
    damage_range: Range.t
  }
  defstruct name: nil,
            description: nil,
            hit_points: 0,
            max_hit_points: 0,
            attack_description: nil,
            damage_range: nil

  defimpl String.Chars do
    def to_string(character), do: character.name
  end
end

# And after they are defined, we can use them like:
defmodule DungeonCrawl.Room.Trigger do
  @callback run(Character.t, Action.t) :: {Character.t, atom}
end
```

- For Dialyzer you need to add a dependency: `{:dialyxir, "~> 0.5", only: [:dev], runtime: false},` (now version 1.4)
- Run `mix do deps.get, deps.compile` after updating dependencies.

- dialyxir adds a `mix dialyzer` command.
- You can make a copy of a map with an updated key with the syntax: `%{character | hit_points: new_hit_points}`.
- The `DungeonCrawl.Battle` module illustrates putting concepts together:

```elixir
defmodule DungeonCrawl.Battle do
  alias DungeonCrawl.Character
  alias Mix.Shell.IO, as: Shell

  def fight(
    char_a = %{hit_points: hit_points_a},
    char_b = %{hit_points: hit_points_b}
  ) when hit_points_a == 0 or hit_points_b == 0, do: {char_a, char_b}
  def fight(char_a, char_b) do
    char_b_after_damage = attack(char_a, char_b)
    char_a_after_damage = attack(char_b_after_damage, char_a)
    fight(char_a_after_damage, char_b_after_damage)
  end

  defp attack(%{hit_points: hit_points_a}, character_b)
    when hit_points_a == 0, do: character_b
  defp attack(char_a, char_b) do
    damage = Enum.random(char_a.damage_range)
    char_b_after_damage = Character.take_damage(char_b, damage)

    char_a
    |> attack_message(damage)
    |> Shell.info

    char_b_after_damage
    |> receive_message(damage)
    |> Shell.info

    char_b_after_damage
  end

  defp attack_message(character = %{name: "You"}, damage) do
    "You attack with #{character.attack_description} " <>
    "and deal #{damage} damage."
  end
  defp attack_message(character, damage) do
    "#{character.name} attacks with " <>
    "#{character.attack_description} and " <>
    "deals #{damage} damage."
  end

  defp receive_message(character = %{name: "You"}, damage) do
    "You receive #{damage}. Current HP: #{character.hit_points}."
  end
  defp receive_message(character, damage) do
```

```
    "#{character.name} receives #{damage}. " <>
    "Current HP: #{character.hit_points}."
  end
end
```

- Protocols vs Behaviours: Protocols work with structs, and behaviours work with modules. Protocols create a function interface to work with several data types. Behaviours define a list of functions that a module should implement.
- DungeonCrawl is a pretty simple module, but it has `DungeonCrawl`, `DungeonCrawl.CLI`, `DungeonCrawl.Room`, `DungeonCrawl.Room.Triggers`, and `Mix.Tasks` namespaces.

Chapter 7 - Handling Impure Functions

- Pure functions are simple to maintain because they are predictable. Impure functions are necessary to build useful software.
- Three ways we will see to deal with unexpected results is with control flow, exceptions (try/catch), and the error monad. We have examples of each in `ch07/dc_flow`, `ch07/dc_try`, `ch07/dc_monad`, and `ch-7/dc_with`. They are all just copies of `dungeon_crawler` that handle errors different ways.
- Here's a simple example of using flow control for errors:

```elixir
defmodule Shop do
  def checkout(price) do
    case ask_number("Quantity?") do
      :error -> IO.puts("It's not a number")
      {quantity, _} -> quantity * price
    end
  end

  def ask_number(message) do
    message <> "\n"
    |> IO.gets
    |> Integer.parse
  end
end

# But imagine we want a quantity and price entered.
def checkout() do
  case ask_number("Quantity?") do
    :error ->
      IO.puts("It's not a number")
    {quantity, _} ->
      case ask_number("Price?") do
        :error ->
          IO.puts("It's not a number")
        {price, _} ->
          quantity * price
      end
  end
end

# We can reduce the noise by using functions.
def checkout() do
  quantity = ask_number("Quantity?")
  price = ask_number("Price?")
  calculate(quantity, price)
end

def calculate(:error, _), do: IO.puts("Quantity is not a number")
def calculate(_, :error), do: IO.puts("Price is not a number")
def calculate({quantity, _}, {price, _}), do: quantity * price
```

- Most of the time, functions that can raise an exception on failure instead of returning an error tuple have names that end in an exclamation point like `File.cd!/1`.

- Elixir supports both `raise/rescue` and `try/throw/catch`.

```
def checkout() do
  try do
    {quantity, _} = ask_number("Quantity?")
    {price, _} = ask_number("Price?")
    quantity * price
  rescue
    MatchError -> "It's not a number"
  end
end

@invalid_option {:error, "Invalid option"}

def parse_answer(answer) do
  case Integer.parse(answer) do
    :error ->
      throw @invalid_option
    {option, _} ->
      option - 1
  end
end

def find_option_by_index(index, options) do
  Enum.at(options, index) || throw @invalid_option
end

def ask_for_option(options) do
  # the try do is strictly optional for a single try block
  try do
    options
    |> display_options
    |> generate_question
    |> Shell.prompt
    |> parse_answer
    |> find_option_by_index(options)
  catch
    {:error, message} ->
      display_error(message)
      ask_for_option(options)
  end
end
```

- Elixir developers normally avoid the additional complexity of using `catch/rescue/raise/throw` to handle exceptional results.
- We're not going to get into the theory of monads. In general, a monad wraps a value with properties that give more information about that value - they give the context. Having a value with context makes it possible for the process of combining functions with values to make automatic decisions. For example, if we have context for when a value is an error or a success, we can automatically skip function executions when the value is an error. When using a monad, you call `bind` and in the context of an error or success, `bind` executes the function passed only if the data is marked as success. When it's a failure, `bind` ignores the function invocation. Elixir doesn't natively support monads, so we're going to use MonadEx library.

```
> iex -S mix
Erlang/OTP 29 [erts-17.0.3] [source] [64-bit] [smp:8:8] [ds:8:8:10] [async-
threads:1] [dtrace]

Compiling 16 files (.ex)
Generated dungeon_crawl app
Interactive Elixir (1.20.2) - press Ctrl+C to exit (type h() ENTER for help)
iex(1)> use Monad.Operators
Monad.Operators
iex(2)> import Monad.Result
Monad.Result
iex(3)> success(42) ~>> (& &1 + 1) ~>> (& &1 + 2)
45
iex(4)> error("wrong") ~>> (& &1 + 1) ~>> (& &1 + 2)
%Monad.Result{type: :error, value: nil, error: "wrong"}
```

- In the above, we used `Monad.Operators` to get the bind operator (~>>). The left side expects a monad and the right side expects a function. We also import `Monad.Result`; the `Result` monad that holds a success or an error. Monads let us complete a full pipeline and check for errors at the end.

```elixir
defmodule DungeonCrawl.CLI.BaseCommands do
  use Monad.Operators

  alias Mix.Shell.IO, as: Shell
  import Monad.Result, only: [success: 1, success?: 1, error: 1, return: 1]

  def display_options(options) do
    options
    |> Enum.with_index(1)
    |> Enum.each(fn {option, index} ->
      Shell.info("#{index} - #{option}")
    end)

    return(options)
  end

  def generate_question(options) do
    options = Enum.join(1..Enum.count(options), ",")
    "Which one? [#{options}]\n"
  end

  def parse_answer(answer) do
    case Integer.parse(answer) do
      :error -> error("Invalid option")
      {option, _} -> success(option - 1)
    end
  end

  def find_option_by_index(index, options) do
    case Enum.at(options, index) do
      nil -> error("Invalid option")
      chosen_option -> success(chosen_option)
    end
  end

  def ask_for_option(options) do
    result =
      return(options)
      ~>> (&display_options/1)
      ~>> (&generate_question/1)
      ~>> (&Shell.prompt/1)
      ~>> (&parse_answer/1)
      ~>> (&(find_option_by_index(&1, options)))

    if success?(result) do
```

```
      result.value
    else
      display_error(result.error)
      ask_for_option(options)
    end
  end

  def display_error(message) do
    Shell.cmd("clear")
    Shell.error(message)
    Shell.prompt("Press Enter to continue.")
    Shell.cmd("clear")
  end
end
```

- Another option is Elixir's `with` special form. If all clauses match, the code executes and returns the `do` block result. If one clause doesn't match, the code stops and returns the value of the non-matching clause. `with` works well for function pipelines that can result in an error. Elixir will first execute the code on the right of the `<-` operator. Then if the pattern matches on the left, Elixir will execute the next instruction. You can add many instructions inside of `with` by separating them with commas. `with` will stop if one of the instructions don't match. The `do` block at the end is the final execution. You can also use an `else` block with `with`.

```
def checkout() do
  result =
    with {quantity, _} <- ask_number("Quantity?"),
         {price, _} <- ask_number("Price?"),
      do: quantity * price
  if result == :error, do: IO.puts("It's not a number"), else: result
end

# with an else clause
def checkout() do
  with {quantity, _} <- ask_number("Quantity?"),
       {price, _} <- ask_number("Price?") do
    quantity * price
  else
    :error ->
      IO.puts("It's not a number")
  end
end
```

- The advantage of `with` is flexibility, but the disadvantage is that it cannot be used with the pipe operator.
- Here's a clean example of `with` how you might see it in the wild:

```elixir
with {:ok, hero} <- HeroChoice.find_hero(choice),
     {:ok, updated_hero} <- Battle.resolve_damage(hero, damage),
     {:ok, saved_hero} <- Repo.save(updated_hero) do
  # The "Return" or "Pure" step of the monad if everything succeeded
  {:ok, saved_hero}
else
  # Catching the "Failure" state (like Left or Nothing)
  {:error, :not_found} -> "Hero went missing!"
  {:error, reason} -> "Failed because: #{reason}"
end
```